

Tipovi podataka, ulaz i izlaz i naredba if

program koji odlučuje (pravi selekciju)

Šta nam nedostaje da prevedemo dijagram za ventilator u program program

Tip za temperaturu

Promjenljiva **t** čuva vrijednost kao što je 23.5. Tip **int** to ne može, pa nam treba tip za realne brojeve.

Način da se učitava

Dijagram kaže *ulaz: t*. U jeziku C to radi naredba **scanf**, uz znak & koji smo prošli put uzeli kao pravilo.

Naredba koja odlučuje

Iz romba izlaze dvije grane. Program mora imati naredbu koja bira kojom će poći: to je **if naredba**.

Na kraju ovog časa taj dijagram postaće program u jeziku C.

Tipovi podataka

Tip	Šta čuva	Primjer	Gdje ga koristimo
int	cijele brojeve	245	broj koraka motora, brojač
float	realne brojeve	23.5	očitana temperatura
double	realne brojeve veće tačnosti	23.456789	proračun u regulaciji
char	jedan znak	'A'	oznaka režima rada

Tip promjenljive biramo prema onome šta je ta veličina, tj. kakve vrijednosti će se čuvati u toj promjenljivoj

Broj koraka motora ne može biti 2.5, a temperatura može. Deklaracija **float t;** kaže prevodiocu koliko memorije da odvoji za promjenljivu t i da tumači kao realni broj ono što se u njoj nalazi.

Deklaracija promjenljive, ime promjenljive i komentari u kodu

Deklaracija promjenljive ide obavezno prije prve upotrebe te promjenljive

int n;

float t, p;

Svaka promjenljiva ima tačno jedan tip i on se ne mijenja do kraja programa.

Imena promjenljivih

- Sastoje se od slova, cifara i donjih crta
- Ime promjenljive ne može počinjati cifrom
- Razlikuju se mala i velika slova, pa su **t** i **T** dvije različite promjenljive.
- Riječi rezervisane za jezik (knjučne riječi) ne mogu biti imena promjenljivih: **int, float, double, char, if, else, while, for, return.**
- Ime promjenljive ne može da se mijenja kroz program

Komentari

Tekst između **/* i */**, ili poslije **//** do kraja reda, prevodilac preskače. To su komentari. Pišu se za čovjeka/programera koji će kod čitati kasnije.

Promjenljiva u memoriji

Svaka promjenljiva ima ime, tip, adresu, veličinu koju zauzima u memoriji i vrijednost koja je u nju upisana

Stanje u memoriji prije naredbe: $\text{zbir} = \text{prvi} + \text{drugi}$;

prvi

45

drugi

72

zbir

—

Stanje u memoriji poslije naredbe: $\text{zbir} = \text{prvi} + \text{drugi}$;

prvi

45

drugi

72

zbir

117

Svaka dodjela vrijednosti nekoj promjenljivoj (upis) briše prethodnu vrijednost koja je bila upisana. Čitanje promjenljive ne mijenja vrijednost promjenljive, pa u našem primjeru promjenljive **prvi** i **drugi** ostaju isti i poslije naredbe.

Ispis: printf i specifikatori

```
int n = 245; // broj koraka
float t = 23.5; /* temperatura */

printf("Koraka: %d\n", n);
printf("Temperatura: %f\n", t);
printf("Temperatura: %.1f C\n", t);
```

```
ispis
Koraka: 245
Temperatura: 23.500000
Temperatura: 23.5 C
```

Specifikator bira tip

%d je specifikator za cio broj, **%f** za realan broj, **%c** za znak. Oznaka **%.1f** traži jednu decimalu, a **\n** prelazak u novi red.

Ako se ne slažu

warning: format '%d' expects argument of type 'int',
but argument 2 has type 'double'
Ispis je tada često besmislen broj.

Ovaj tekst prevodilac preskače, pišemo ga za čovjeka koji će čitati kod kasnije

Ulaz: scanf i znak &

```
float t;  
  
printf("Temperatura: ");  
scanf("%f", &t);
```

Zato je & obavezno

Naredbi **scanf** ne predajemo vrijednost promjenljive **t** nego mjesto na koje **scanf** treba da upiše ono što pročita sa tastature. Znak & ispred imena daje baš to: **adresu** promjenljive **t**.

Ako korisnik treba da unese vrijednosti za više promjenljivih (npr. tri vrijednosti za temperaturu) i to želimo da omogućimo u jednoj liniji koda:

```
scanf("%f%f%f", &t1, &t2, &t3);
```

Korisnik unosi vrijednosti i razdvaja ih razmakom ili tasterom Enter.

Ako se & izostavi

warning: format '%f' expects argument of type 'float *', but argument 2 has type 'double'

Program se prevede, ali pri pokretanju najčešće puca, jer upisuje na adresu koja ne postoji.

I u scanf kao u printf specifikator prati tip

%d učitava cio broj, **%f** realan.

Dijeljenje, ostatak, prioritet i zagrade

Izraz	Rezultat	Obrazloženje
$7 / 5$	1	oba operanda su cijeli brojevi, pa se decimalni dio odbacuje
$7 \% 5$	2	ostatak pri dijeljenju cijelih brojeva
$7.0 / 5$	1.4	jedan operand je realan, pa je i rezultat realan
$7.5 \% 2$	greška	operator % radi samo sa cijelim brojevima

Prioritet operatora

Množenje i dijeljenje idu prije sabiranja, kao u matematici. Zato $1 + 2 + 6 / 3$ daje **5**, a $(1 + 2 + 6) / 3$ daje **3**.

Poređenje vraća običan broj (0 ili 1)

==

!=

<

>

<=

>=

Poređenje daje 1 ili 0

Naredba `printf("%d\n", 5 > 3);` ispisuje **1**. Tačno je 1, netačno je 0.

Nula je netačno, sve ostalo je tačno

U jeziku C ne postoji poseban tip za tačno i netačno. Zato je **if (t)** ispravan zapis, a znači: ako **t** nije nula.

Naredba if

jedna naredba u grani

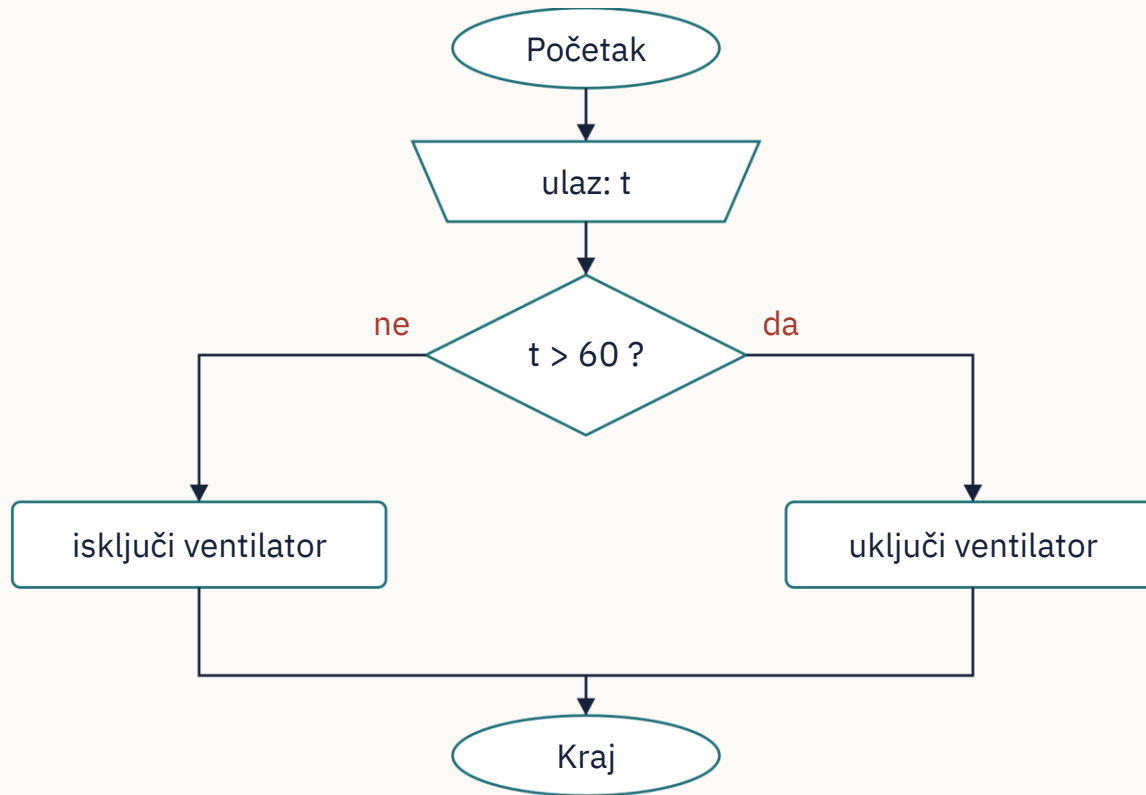
```
if (t > 60)
    printf("Ukljuci ventilator\n");
```

dvije grane i više naredbi

```
if (t > 60) {
    printf("Ukljuci ventilator\n");
    stanje = 1;
} else {
    printf("Iskljuci ventilator\n");
    stanje = 0;
}
```

Bez vitičastih zagrada uz **if** ide samo jedna naredba. Kada ih ima više, zagrade su obavezne.

Ventilator: od dijagrama do programa



```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    float t;
```

```
    printf("Temperatura: ");
```

```
    scanf("%f", &t);
```

```
    if (t > 60)
```

```
        printf("Ukljuci ventilator\n");
```

```
    else
```

```
        printf("Iskljuci ventilator\n");
```

```
    return 0;
```

```
}
```

Svaki simbol dijagrama ima svoj red u programu. Obje grane vode u isti kraj, pa i program ima samo jedan **return**.

Tri greške koje prevodilac ne prijavljuje a program neće raditi dobro

Dodjela umjesto poređenja

```
if (t = 60)
```

Promjenljiva **t** dobija vrijednost 60, a uslov je uvijek tačan.

Tačka-zarez iza zagrade uslova

```
if (t > 60);
```

Naredba **if** tu se završava. Ono što slijedi izvršava se uvijek i ne zavisi od ispunjenosti uslova

Dvije naredbe bez zagrada

```
if (t > 60)  
    printf("Ukljuci\n");  
    stanje = 1;
```

Grani pripada samo prvi red; drugi se izvršava bez obzira na ispunjenost uslova.

Zadaci

pišu se u CodeBlocks-u, prevode i pokreću

Zadatak 1

Učitati temperaturu i ispisati upozorenje ako pređe 80 stepeni.

Zadatak 2

Učitati dva cijela broja i ispisati veći.

Zadatak 3

Učitati cio broj i ispisati da li je paran. Koristiti operator %.

Zadatak 4

Učitati tri ocjene i ispisati prosjek sa jednom decimalom.